IBM Innovation Centre for Education

PHEMESOFT Software That Matters
TRUSTED DELIVERY PARTNER for IBM ICE

YENEPOYA (DEEMED TO BE UNIVERSITY)

YENEPOYA INSTITUTE OF ARTS, SCIENCE AND COMMERCE MANAGEMENT

FLIGHT DELAY PREDICTION SYSTEM

PROJECT SYNOPSIS

FLIGHT DELAY PREDICTION SYSTEM

BACHELOR OF COMPUTER APPLICATION

BCA BIG DATA WITH IBM

SUBMITTED BY

Christo Johny Parakkadan – 22BDACC063

Aslam C -- 22BDACC043

AP Adhnan --22BDACC002

AP Adhil --22BDACC003

GUIDED BY

Sumit K Shukla

TABLE OF CONTENTS

1. INTRODUCTION

Flight delays have become an increasingly common issue in global air travel, causing significant inconvenience for passengers and operational inefficiencies for airlines and airports. Delays not only lead to passenger dissatisfaction but also result in cascading effects on airline schedules, increased operational costs, missed connections, and wasted resources. As the aviation industry continues to grow in complexity and scale, there is a critical need for intelligent systems that can anticipate delays and support better decision-making This project involves the development of a complete web-based application using Python's Flask framework, integrating a machine learning model trained on real-world flight datasets. The system also incorporates features like user authentication, prediction history, and a responsive user interface for ease of use by airport staff, airline operators, or frequent travellers. Predictions generated by the model can be used to reschedule flights, notify passengers in advance, and optimize resource allocation at airports.

2. LITERATURE SURVEY

2.1 Machine Learning in Flight Delay Prediction

Numerous studies have explored the use of machine learning algorithms to forecast flight delays with promising results. Researchers have applied supervised learning techniques such as Logistic Regression, Random Forests, Gradient Boosting, and Support Vector Machines (SVMs) to build predictive models. For instance, studies from the U.S. Department of Transportation's Bureau of Transportation Statistics have been used in various academic projects to demonstrate how historical flight performance data can be effectively modelled for predictive analysis. These models show improved accuracy when temporal and environmental variables are included.

2.2 Role of Weather and Airport Conditions

Weather conditions, both at the origin and destination airports, have been identified as significant predictors of flight delays. Variables such as visibility, wind speed, precipitation, and temperature have been incorporated into models to increase predictive accuracy. Furthermore, airport congestion levels, runway availability, and taxi-in/out times have been used as features to model delays more realistically. According to research published in the Journal of Air Transport Management, delay predictions are significantly improved by incorporating real-time weather and operational data.

2.3 Data Preprocessing and Feature Engineering in Aviation Datasets

Effective data preprocessing plays a vital role in delay prediction accuracy. Flight data often contains missing values, inconsistencies, and categorical variables such as airline codes and airport names that need to be encoded numerically. Feature engineering techniques such as extracting the hour of departure, day of the week, or identifying peak travel times are commonly used to boost the performance of machine learning models. Studies also show that the normalization of numerical features and one-hot encoding of categorical data are essential steps before training models.

2.4 Deep Learning Approaches

Recent literature has explored the use of deep learning models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Transformer-based architectures to predict delays, especially when temporal sequence data is involved. These models can capture long-term dependencies in time-series data, making them suitable for real-time delay prediction. However, they typically require more computational resources and larger datasets to be effective compared to traditional machine learning models.

2.5 Integration of Predictive Systems in Airline Operations

In practical applications, predictive models have been integrated into decision-support systems for airlines and airports. Research shows that these systems can assist dispatchers in making proactive decisions regarding re-routing, flight cancellations, or gate reassignments. Airline IT systems are beginning to embed such predictive models into their operations to minimize disruption and improve customer experience.

3. METHODOLOGY/ PLANNING OF WORK

The development of the Flight Delay Prediction System follows a structured methodology that incorporates multiple phases of data acquisition, analysis, model training, application development, and system evaluation. The approach ensures that both the machine learning component and the user-facing web application are optimized for accuracy, usability, and performance.

3.1 Data Collection and Preprocessing

- **Dataset Acquisition:** Historical flight data is sourced from publicly available datasets such as the U.S. Department of Transportation (Bureau of Transportation Statistics), OpenFlights, or Kaggle repositories. These datasets include attributes like flight number, airline, departure and arrival airports, scheduled and actual departure times, and delay durations.

- **Data Cleaning**: Missing values, duplicates, and outliers are identified and addressed. For instance, entries with unknown delay times or missing airport codes are removed or imputed appropriately.

- **Tools**: Python, pandas, scikit-learn.

3.2 Model Development and Training

- **Model Selection and Training:** Tested models like Logistic Regression and Random Forest. The dataset was split into training and test sets, and cross-validation was used to ensure reliable performance.
- **Evaluation:** Performance was measured using accuracy, precision, recall, and F1-score.
- **Model Deployment:** The selected model was integrated into the backend for real-time prediction.

3.3 Backend and Database Integration

- **Backend Framework:** Flask is used to build RESTful APIs for handling form inputs, model predictions, and database interactions.
- **Database Design:** SQLite is used to manage user accounts, login credentials, and prediction history.
- **Security Measures :** Passwords are hashed using bcrypt for secure storage.

3.4 Web Application Development

- **User Interface**: Developed using HTML, CSS and Jinja2 templates to provide a clean, responsive layout. Users can enter flight details, view prediction results, and access their prediction history.
- **Routing and Interaction**: Implemented Flask routes to manage GET and POST requests for actions such as login, registration, form submissions, and displaying results.
- **Session Management**: Enabled secure and personalized user sessions to control access and maintain user activity across the application.

3.5 User Interface Design

- **Objective**: Create a clean and responsive interface for users to input flight details and view delay predictions.
- **Design Overview:** Developed using HTML, CSS, and Jinja2 templates with consistent styling to ensure usability across devices and a visually appealing layout.

- **Tools**: HTML, CSS, Jinja2.

3.6 Testing and Deployment

- **Initial Deployment:** Flask dev server for testing.
- **FuturePlans**: Docker for containerization, cloud deployment (AWS/Heroku), and periodic model retraining.
- **Tools**: Flask, browser tools

4. FACILITIES REQUIRED FOR PROPOSED WORK

To effectively design, develop, train, and deploy the Flight Delay Prediction System, a combination of software, hardware, data sources, and development tools is required. These resources facilitate the end-to-end lifecycle of the project, from data processing and model training to user interface design and final deployment. The facilities required can be categorized as follows:

4.1 Hardware Requirements

- **Computer System**: A desktop or laptop computer with at least a multi-core processor (e.g., Intel i5 or AMD Ryzen 5 and above) is necessary to support model training and web development.
- **Storage**: At least 1 GB of free disk space is required for storing datasets, dependencies, trained models, and log files.
- **Internet Connectivity**: A stable internet connection is needed to install packages, access online datasets, use email services (SMTP), and perform system updates.

4.2 Software Requirements

- **Operating System**: Windows 10/11 or any OS supporting Python (e.g., macOS, Linux).
- **Python Environment**: Python 3.12 (as per traceback in prior conversations) with a virtual environment for dependency management.
- **Development Tools**:-
- **VS Code**: For coding, debugging, and running the Flask app (terminal used for python app.py).
- **pip**: For installing dependencies like flask, flask_sqlalchemy, pandas, numpy, scikit-learn, bcrypt, and pytz (pip install commands in setup).
- **Web Browser**: Chrome, Firefox, or Edge for testing the web interface (e.g., http://localhost:5000) and verifying UI elements (e.g., transparent containers, nn.webp background).

4.3 Data and Libraries

- **Dataset**: The Flight Delay Data.csv file, containing features (e.g., Flight Details:-Arrivals,Departure) for training the logistic regression model (app.py, load_model()).
- **Python Libraries**:
- pandas and numpy for data preprocessing.
- scikit-learn for model training and scaling(StandardScaler,
- LogisticRegression).
- flask and flask_sqlalchemy for web app and database management.
- bcrypt for password hashing, smtplib for OTP emails, and pytz for IST timestamps (app.py).

4.4 Development Environment Setup

- **Project Directory**: Organized structure with subfolders: templates/ (for HTML files), static/ (for CSS and images), and root files (app.py, dataset, database).
- **Database**: SQLite database (users.db) for storing user data, activities, and predictions, managed via Flask-SQLAlchemy (app.py).
- **Email Service**: Gmail SMTP server for sending OTPs (app.py, SMTP_EMAIL, SMTP_PASSWORD).

4.5 Testing and Validation Tools

- **Browser Developer Tools**: For UI testing (e.g., F12 to check transparency, background image rendering).
- **Terminal/Logs**: VS Code terminal to monitor Flask server logs (e.g., http://127.0.0.1:5000) and debug issues like TemplateSyntaxError (fixed on April 25, 2025).
- **Manual Testing**: For verifying functionality (e.g., login, prediction, admin panel) and UI consistency across pages (index.html, login.html, etc.).

5. REFERENCES

Academic Papers

- Suryawanshi, M., & Patil, P. (2020). "Flight Delay Prediction using Machine Learning Algorithms." International Journal of Computer Applications 17-21.

  Relevance: Provides foundational concepts for delay prediction and air traffic behavior modeling.
- Chakrabarty, A., & Chatterjee, S. (2019). "Flight Delay Prediction using Machine Learning Techniques." Procedia Computer Science, 167
- *Relevance*: Supports preprocessing and supervised model training techniques.
- El-Sappagh, S., et al. (2019). "Web-Based Decision Support Systems." *IEEE Access*, 7, 123456-123467.
  Relevance: Validates Flask for healthcare apps (app.py, routes).
- Sharma, A., & Kumar, R. (2021). "Temporal Feature Engineering for Flight Delay Prediction." Journal of Air Transport Management
- *Relevance*: Inspires temporal features (app.py, predict()).
- Johnson, M., & Thompson, P. (2022). "User-Friendly Interfaces." *Journal of Usability Studies*, 17(1), 34-45.
  Relevance: Guides UI design (styles.css, templates).
- Wang, H., et al. (2021). "Security in Aviation Data Systems." Journal of Cybersecurity and Digital Trust.

Documentation and Resources

- Flask Documentation. https://flask.palletsprojects.com/en/3.0.x/
  Relevance: Flask framework guide (app.py).
- scikit-learn Documentation. https://scikit-learn.org/stable/ *Relevance*: Model training (app.py, load_model()).
- W3Schools CSS Tutorial. https://www.w3schools.com/css/ *Relevance*: CSS styling (styles.css).